

Integrating OpenCode as a Backend for MCP

Interface: A Technical Deep Dive

- OpenCode in `opencode web` mode maintains isolated sessions with unique IDs, each with independent workspace state.
- The OpenCode Web API provides extensive endpoints for session lifecycle, file handling, and task execution, optimized for remote interactions.
- Sessions are created and managed via REST endpoints, with support for hierarchical relationships and state persistence.
- File uploads and context sharing are handled via multipart requests and URI-based references, enabling seamless integration with MCP hosts.
- Architectural recommendations include session isolation, URI-based context passing, and careful handling of concurrency and synchronization.

Introduction

The Multi-Agent Communication Protocol (MCP) enables AI agents to interact with external tools and services through a standardized interface, extending the capabilities of large language models (LLMs) beyond their native context. OpenCode, a powerful AI coding agent, exposes an HTTP API in its `opencode web` mode designed for remote and web-based interactions. This report provides a detailed technical analysis of OpenCode's session and workspace isolation model, its Web API endpoints relevant to MCP integration, URI and context handling mechanisms, and architectural recommendations for using OpenCode as a backend MCP host. The goal is to replace a closed-source MCP host lacking subagent features with OpenCode, enabling code-related operations via its web API.

Session and Workspace Isolation Model

OpenCode's `opencode web` mode is designed to support multiple concurrent sessions, each isolated with its own workspace state. This isolation is critical for enabling multiple MCP agents or clients to interact with OpenCode without conflicts.

Session Isolation

- Each session is assigned a unique session ID upon creation, which serves as the primary identifier for all operations within that session.
- Sessions maintain independent state including open files, terminal history, model memory, and workspace context.
- The session lifecycle is managed through REST endpoints, enabling creation, updates, sharing, and cleanup of sessions.



- Sessions support hierarchical relationships via `parentID`, allowing child sessions to inherit context from parent sessions, useful for managing complex workflows.

Workspace Model

- Each session operates within its own workspace, which includes a filesystem, Git repository, and loaded models.
- File changes and workspace state are tracked independently per session, preventing conflicts between concurrent users.
- The workspace is initialized and managed via API calls, enabling the MCP host to upload context (e.g., Git repositories or config files) specific to a session.

Concurrency and Risks

- Since sessions are isolated, concurrent modifications by multiple agents are naturally scoped to their respective sessions, minimizing risks of git conflicts, file overwrites, or model state corruption.
- However, if multiple agents share the same session ID, race conditions may occur. The MCP host must ensure unique session IDs are used per agent or implement locking mechanisms.
- Performance considerations include model loading overhead per session; shared model instances or caching strategies may be needed to optimize resource usage.

Web API Reference

The OpenCode Web API in **opencode web** mode exposes a rich set of endpoints optimized for remote interactions. Below is a detailed table of the most relevant endpoints for MCP integration:

Session Management

Method	Path	Description	Parameters / Notes
GET	<code>/session</code>	List all sessions	Returns array of Session objects
POST	<code>/session</code>	Create a new session	Body: { <code>parentID?</code> , <code>title?</code> }, returns Session
GET	<code>/session/:id</code>	Get session details	Returns Session object
DELETE	<code>/session/:id</code>	Delete session and all its data	Returns boolean
PATCH	<code>/session/:id</code>	Update session properties	Body: { <code>title?</code> }, returns Session
POST	<code>/session/:id/fork</code>	Fork session at a message	Body: { <code>messageID?</code> }, returns Session



Method	Path	Description	Parameters / Notes
POST	/session/:id/abort	Abort running session	Returns boolean
POST	/session/:id/share	Share session	Returns Session
DELETE	/session/:id/share	Unshare session	Returns Session
GET	/session/:id/diff	Get diff for session	Query: messageId?, returns FileDiff[]
POST	/session/:id/ summarize	Summarize session	Body: { providerID, modelID }, returns boolean

File and Context Handling

Method	Path	Description	Parameters / Notes
GET	/find?pattern=<pat>	Search for text in files	Returns array of match objects
GET	/find/file?query=<q>	Find files/directories by name	Returns string[] (paths)
GET	/find/symbol? query=<q>	Find workspace symbols	Returns Symbol[]
GET	/file?path=<path>	List files and directories	Returns FileNode[]
GET	/file/content? path=<p>	Read file content	Returns FileContent
GET	/file/status	Get status for tracked files	Returns File[]

Task Execution

Method	Path	Description	Parameters / Notes
POST	/session/:id/init	Analyze app and create AGENTS.md	Body: { messageId, providerID, modelID }, returns boolean
POST	/session/:id/message	Send message and wait for response	Body: { message }, returns response
POST	/session/:id/revert	Revert a message	Body: { messageId, partID? }, returns boolean
POST	/session/:id/ permissions/:permissionID request	Respond to permission request	Body: { response, remember? }, returns boolean



Authentication and Configuration

- OpenCode supports OAuth authentication for remote MCP servers, handling token storage and automatic OAuth flows.
- MCP servers are configured in `opencode.json`, specifying type (local/remote), command, environment variables, and headers.

URI and Context Handling

OpenCode supports URI schemes for referencing external resources, enabling seamless integration with MCP hosts and external tools:

- **URI Schemes:** The API supports `file://`, `git://`, and potentially custom `opencode://` URIs for referencing files, repositories, and other resources.
- **Context Passing:** File paths and URIs can be passed in API requests to reference external resources, enabling the MCP host to upload and manage context efficiently.
- **Integration Examples:** OpenCode can integrate with tools like `lazygit` or `vim` via URI passing, allowing external editors to open and modify files referenced in OpenCode sessions.

Architectural Recommendations for MCP-OpenCode Bridge

Sequence Diagram

```
sequenceDiagram
    participant MCP as MCP Host
    participant OC as OpenCode Server

    MCP->>OC: POST /session (create new session)
    OC-->>MCP: Session ID
    MCP->>OC: POST /session/:id/init (analyze app, create AGENTS.md)
    OC-->>MCP: Success
    MCP->>OC: POST /session/:id/message (send task request)
    OC-->>MCP: Response (streams or polling)
    MCP->>OC: GET /session/:id/diff (retrieve changes)
    OC-->>MCP: FileDiff[]
    MCP->>OC: DELETE /session/:id (cleanup session)
    OC-->>MCP: Success
```

Risk Mitigation Strategies

- **Session Isolation:** Ensure each MCP agent uses a unique session ID to avoid conflicts. Implement session locking if multiple agents access the same session.
- **Concurrency Control:** Use version control (e.g., Git) within sessions to manage file changes and avoid conflicts.



- **Performance Optimization:** Cache model instances and session state where possible to reduce overhead.
- **Data Synchronization:** Implement robust protocols to synchronize changes between the MCP host and OpenCode, especially when both modify files.

Open Questions and Gaps

- **Concurrent Modifications:** The exact mechanisms for handling concurrent file modifications and ensuring consistency in multi-agent environments need further investigation.
- **URI and Callback Integration:** While URI schemes are supported, concrete examples of integrating OpenCode with external tools via URIs or callbacks are limited and require experimentation.
- **Authentication and Security:** OAuth support is documented, but the security implications and best practices for MCP server authentication in production environments need deeper exploration.

Summary Table of Key OpenCode Web API Endpoints for MCP Integration

Category	Endpoint	Method	Description	Notes
Session Management	/session	GET	List all sessions	Returns Session[]
	/session	POST	Create new session	Body: { parentID?, title? }
	/session/:id	GET	Get session details	Returns Session
	/session/:id/fork	POST	Fork session at message	Body: { messageID? }
	/session/:id/abort	POST	Abort running session	Returns boolean
	/session/:id/share	POST	Share session	Returns Session
	/session/:id/diff	GET	Get session diff	Query: messageID?
File Handling	/find?pattern=<pat>	GET	Search text in files	Returns match objects
	/find/file?query=<q>	GET	Find files/directories by name	Returns string[] (paths)
	/file?path=<path>	GET	List files and directories	Returns FileNode[]
	/file/content?path=<p>	GET	Read file content	Returns FileContent



Category	Endpoint	Method	Description	Notes
Task Execution	/session/:id/init	POST	Analyze app and create AGENTS.md	Body: { messageId, providerID, modelID }
	/session/:id/message	POST	Send message and wait for response	Body: { message }
	/session/:id/revert	POST	Revert a message	Body: { messageId, partID? }

Conclusion

OpenCode's **opencode web** mode provides a robust, isolated session and workspace model well-suited for integration as an MCP backend. Its comprehensive Web API supports session lifecycle management, file handling, and task execution, enabling MCP hosts to delegate code-related operations efficiently. By leveraging session isolation, URI-based context passing, and careful concurrency control, OpenCode can replace closed-source MCP hosts lacking subagent features, providing a flexible and scalable backend for multi-agent communication protocols.

The integration design should emphasize session isolation, secure authentication, and synchronization mechanisms to mitigate risks of concurrent modifications. Further experimentation is needed to fully explore URI-based integration with external tools and to refine the handling of complex multi-agent workflows.

This technical foundation enables the MCP host to task OpenCode with code generation, editing, and analysis operations via its web API, unlocking powerful AI-driven coding assistance in a multi-agent environment.

